Algorithms Lab

Week 1 – Coding Agents

Korea University Sejong Campus, Department of Computer Science & Software

Lab Overview

- **Goal**: Install and use an AI-powered coding agent for real development tasks
- **Duration**: ~50 minutes
- **Submission**: None – exploration lab
- These tools will be used **throughout the semester** for labs and assignments

Task 1
Install Agent

→

Task 2
File Organization

→

Task 3
Repo Documentation

→

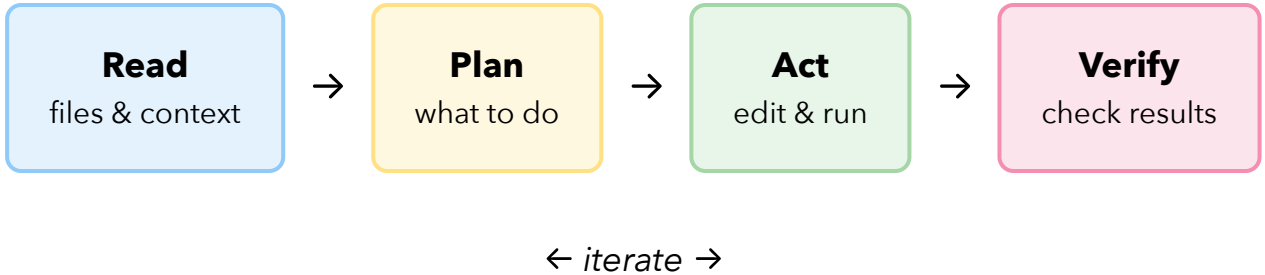
Task 4
RALPH Technique

→

Task 5
Sorting Benchmark

What Are Coding Agents?

AI-powered CLI tools that understand and generate code **in context**



- Can read your file system, run commands, and make edits autonomously
- Useful for: scaffolding, refactoring, documentation, debugging

Example: *"Create a Python sorting benchmark with timing utilities"* Agent: reads directory → writes files → confirms tests pass

Available Agents

Gemini CLI
Free (1000 calls/day)
Google's agent
Good default choice

Claude Code
Paid
Anthropic's agent
Strong multi-file reasoning

Codex CLI
Paid
OpenAI's agent
Open-source CLI

Also available: **OpenCode** (open-source harness – use any model including open-source LLMs)

> *Recommendation*: Install **Gemini CLI** if you have no preference – it's free and easy to set up.

Task 1 – Install a Coding Agent

Install at least one coding agent by following its official documentation.

Installation commands:

```
npm install -g @google/gemini-cli          # Gemini CLI (free)
npm install -g @anthropic-ai/claude-code    # Claude Code (paid)
```

Verify installation:

```
gemini --version    # or: claude --version
```

What to check:

- The CLI launches without errors
- You can authenticate (Google account for Gemini, Anthropic account for Claude)
- Try a simple prompt: `"What is 2 + 2?"` to confirm it responds

Task 2 – Organize Files

Use the agent to **organize files** in a messy directory.

Example prompt:

```
"Organize the files in ~/Downloads by file type into subfolders
 (images, documents, code, etc.). Show me the plan before executing."
```

What to observe:

- Does the agent **ask for confirmation** before moving files?
- Does it create a sensible folder structure?
- Does it handle edge cases (e.g., files with no extension)?

Discussion:

- What would happen if you didn't say *"show me the plan first"*?
- How do you give the agent more specific instructions if the result isn't right?

Task 3 – Document a GitHub Repo

Use the agent to **generate a README.md** for an existing codebase.

Pick a target repository:

- Your own project, or a public repo such as:
  - `https://github.com/code-yeongyu/oh-my-opencode`

Example prompt:

```
"Read this codebase and generate a comprehensive README.md
 with architecture overview, setup instructions, and usage examples."
```

Evaluate the output:

- Does the README **accurately** describe the project?
- Are the setup instructions correct and complete?
- Is anything missing (license, contributing guide, screenshots)?

> Save this README – you'll improve it in the next task.

Task 4 – RALPH Technique

What is RALPH Mode?

An **autonomous verification loop** where coding agents run continuously, self-validating outputs without human intervention.

Request → **A**nalyze → **L**ist issues → **P**rompt again → **H**armonize

> The developer's role is shifting from **implementer** to **architect** – you design specs and validation systems, the agent codes.

RALPH Mode – Real-World Impact

At Korea's first **Ralphton** hackathon, 13 elite teams let AI agents code **autonomously overnight** and reviewed results the next morning.

100,000	70%	133
lines of code (winner)	was test code	Socratic reasoning iterations

The Engineering Evolution

```
Prompt Engineering  →  Context Engineering  →  Harness Engineering
   (what to say)        (what to provide)       (how to verify)
```

Source: Korean First Ralphton Review – Brian Jang

Task 4 – RALPH in Practice

Create a **verifiable evaluation rubric** and use it to iteratively improve output quality.

Example workflow using the README from Task 3:

```
You:    "Generate a rubric for evaluating a high-quality open-source README."
Agent:  Returns 8 criteria (description, install, usage, architecture, ... )

You:    "Now evaluate the README you wrote against this rubric. Score each criterion."
Agent:  Scores 6/8 — missing: architecture diagram, contributing guide.

You:    "Fix all failing criteria. Add an architecture diagram and contributing guide."
Agent:  Updates the README with both additions.

You:    "Re-evaluate. Are all criteria met now?"
Agent:  8/8 — all criteria satisfied.
```

Key phrases to try:

- *"Keep going until the criteria are met"*
- *"Evaluate against the rubric and fix all issues"*

Why this matters:

Task 5 – Build a Sorting Benchmark

Use the agent to **build a multi-file sorting benchmark** – a preview of the semester project.

Example prompt:

```
"Create a Python sorting benchmark that compares bubble sort, merge sort,
 and Python's built-in sort. Include timing utilities and a results table.
 Organize the code into separate modules."
```

Expected output files:

sorting.py
sort algorithms

→

timer.py
timing utility

→

benchmark.py
main runner

→

README.md
documentation

Task 5 – What to Observe

You do **not** need a perfect benchmark – the **process** matters.

Watch how the agent:

- Breaks a complex problem into multiple files
- Explains each component's role
- Handles errors when you give feedback
- Iterates on build failures (if any)

Try follow-up prompts:

- *"Add a bar chart visualization of the results"*
- *"Explain the time complexity of each sorting algorithm"*
- *"The benchmark crashes with large N – fix it"*

Assignments & Team Project

Weekly Assignments (Weeks 02–06)

- **Weeks 2–6**, you will receive weekly assignments
- Each assignment: build a **web application** that applies the algorithm learned that week
- Goal: experience how algorithms improve real software performance

Team Project (Final Exam, Weeks 09–13)

- Teams of 3–4 will build a **web application** that incorporates **all algorithms** covered in the course
- Deliverables: **working web app** + **report** + **presentation**
- The agent will be your primary development tool throughout

> Make the most of coding agents – they will be essential for both assignments and the final project.

Summary & Next Steps

What we practiced today:

Task	Skill Learned
1. Install agent	Tool setup, authentication, basic prompting
2. File organization	Delegating real tasks, reviewing agent decisions
3. Repo documentation	Evaluating AI-generated technical writing
4. RALPH technique	Systematic iterative refinement with rubrics
5. Sorting benchmark	Tackling complex multi-file algorithm projects

Coming up – Week 2 Lab: Complexity Analysis Practice ($O(n^2)$ vs $O(n)$ comparison)

> Coding agents are tools – understanding what they produce is still **your** responsibility.